# Reto completo: del dataset al despliegue de Machine Learning

Google Colab + datos propios + Streamlit + GitHub + Docker + Google Cloud Compute Engine

Modalidad: trabajo en parejas. Cada pareja desarrolla una ruta completa: supervisada o no supervisada. Todo el desarrollo puede realizarse desde el navegador.

Propósito. Construir una solución pequeña pero completa de Machine Learning: cargar datos, preparar variables, entrenar y evaluar un modelo, guardar el pipeline, construir una interfaz Streamlit, empaquetarla con Docker y desplegarla en una máquina virtual de Google Cloud.

No se busca un modelo sofisticado. Se busca que el estudiante entienda el ciclo de vida de una solución y pueda reproducirla en un entorno diferente al de desarrollo.

## 1. ¿Qué aprenderemos?

- Trabajar con un dataset real cargado desde el computador o desde una URL.
- Separar preparación de datos, entrenamiento, evaluación y serving.
- Construir un pipeline de scikit-learn que incluya transformaciones y modelo.
- Serializar el artefacto entrenado con joblib.
- Construir una aplicación Streamlit que reciba nuevos datos.
- Crear archivos independientes: train.py, app.py, requirements.txt, Dockerfile y README.md.
- Comprender la diferencia entre Dockerfile, imagen y contenedor.
- Construir la imagen en una VM Linux y ejecutar el contenedor.
- Exponer la aplicación mediante una IP externa y una regla de firewall.
- Comprobar reproducibilidad mediante una prueba cruzada entre parejas.

## 2. Las dos rutas del proyecto

| Ruta A - Aprendizaje supervisado | Ruta B - Aprendizaje no supervisado |
|---|---|
| Pregunta: ¿podemos predecir una respuesta conocida? | Pregunta: ¿podemos descubrir grupos o perfiles sin una respuesta conocida? |
| Datos: X + variable objetivo y. | Datos: X; no se usa una y para entrenar. |
| Problema sugerido: clasificación binaria o multiclase. | Problema sugerido: segmentación con K-Means. |
| Modelos posibles: regresión logística, árbol, random forest. | Modelo recomendado para despliegue: K-Means, porque permite asignar nuevos casos mediante predict(). |
| Evaluación: matriz de confusión, F1, recall, ROC-AUC, accuracy según el problema. | Evaluación: silhouette, inercia, estabilidad y perfiles promedio de clusters. |
| Salida de la app: clase predicha y, cuando sea posible, probabilidad. | Salida de la app: cluster asignado + descripción interpretable del perfil. |

Importante: las dos rutas deben usar la misma arquitectura de ingeniería. Lo que cambia es el problema estadístico y la interpretación; Streamlit, Docker y GCP siguen prácticamente el mismo flujo.

## 3. Flujo completo del trabajo

```
1. Seleccionar o cargar dataset
          |
2. Explorar y definir variables
          |
3. Crear train.py
          |
4. Entrenar + evaluar
          |
5. Guardar model.joblib
          |
6. Crear app.py
          |
7. Probar Streamlit en Colab
          |
8. Crear requirements.txt + Dockerfile + .dockerignore
          |
9. Guardar proyecto en GitHub
          |
10. Crear VM en GCP
          |
11. Instalar Docker en la VM
          |
12. git clone
          |
13. docker build
          |
14. docker run
          |
15. Abrir firewall
          |
16. Probar IP pública
          |
17. Prueba cruzada con otra pareja
```

Esta secuencia es también el checklist principal de la actividad.

# 4. Preparación del proyecto en Google Colab

Colab será nuestro entorno de desarrollo. No necesitamos VS Code, Python local ni Docker Desktop.

## Paso 4.1 - Instalar dependencias

```
!pip -q install streamlit scikit-learn pandas numpy joblib openpyxl
```

## Paso 4.2 - Crear una carpeta de proyecto

```
from pathlib import Path

PROJECT = Path("/content/ml_streamlit_project")
PROJECT.mkdir(exist_ok=True)
print(PROJECT)
```

Todos los archivos que creemos se guardarán dentro de esta carpeta.

## Paso 4.3 - Recordatorio sobre persistencia

El almacenamiento de Colab es temporal. Un reinicio del runtime puede eliminar archivos. Por eso deben conservar el proyecto en GitHub o descargar una copia antes de finalizar la sesión.

# 5. Trabajar con datos reales o cargados por el estudiante

La práctica no depende de datasets sintéticos. Cada pareja puede trabajar con un CSV o Excel propio, siempre que sea razonablemente pequeño y pueda procesarse con pandas/scikit-learn.

## Opción A - Subir un archivo desde el computador

```
from google.colab import files

uploaded = files.upload()

# Después de seleccionar el archivo:
import pandas as pd

filename = next(iter(uploaded))
print("Archivo cargado:", filename)

if filename.lower().endswith(".csv"):
    df = pd.read_csv(filename)
elif filename.lower().endswith((".xlsx", ".xls")):
    df = pd.read_excel(filename)
else:
    raise ValueError("Use un archivo CSV o Excel.")

df.head()
```

## Opción B - Leer datos desde una URL pública

```
import pandas as pd

url = "https://.../datos.csv"
df = pd.read_csv(url)
df.head()
```

## Opción C - Dataset incluido en scikit-learn

Puede utilizarse para practicar el flujo antes de reemplazarlo por datos propios. No es obligatorio para el proyecto final.

# 6. Revisión mínima del dataset

Antes de modelar, cada pareja debe responder: ¿qué representa una fila?, ¿cuál es el objetivo del análisis?, ¿qué variables pueden entrar al modelo?, ¿hay datos faltantes?, ¿hay variables numéricas y categóricas?

```
print(df.shape)
display(df.head())
display(df.dtypes)
display(df.isna().sum().sort_values(ascending=False).head(15))
```

```
display(df.describe(include="all").T)
```

La aplicación final debe recibir únicamente variables que tengan sentido para un nuevo caso. No incluya identificadores, variables posteriores al evento o información que produzca fuga de información.

# 7. Ruta A - Construcción supervisada completa

Supongamos que el dataset contiene una variable objetivo llamada target. Cada pareja debe reemplazar los nombres por los de su dataset.

## Paso 7.1 - Definir objetivo y variables

```
TARGET = "target"

numeric_features = [
    "edad",
    "ingreso",
    "antiguedad"
]

categorical_features = [
    "canal",
    "region"
]

features = numeric_features + categorical_features

X = df[features].copy()
y = df[TARGET].copy()
```

## Paso 7.2 - Separar entrenamiento y prueba

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

## Paso 7.3 - Construir preprocesamiento + modelo

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)
```

Usar un Pipeline es fundamental: las mismas transformaciones utilizadas durante entrenamiento se aplicarán automáticamente cuando Streamlit reciba un nuevo caso.

# 8. Ruta A - Evaluación y artefacto

## Paso 8.1 - Evaluar

```
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    f1_score
)

pred = model.predict(X_test)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
print("F1:", f1_score(y_test, pred, average="weighted"))
```

La pareja debe explicar por qué la métrica seleccionada es razonable para su problema. Accuracy no siempre es suficiente, especialmente con clases desbalanceadas.

## Paso 8.2 - Guardar metadatos y modelo

```
import joblib

artifact = {
    "model": model,
    "features": features,
    "numeric_features": numeric_features,
    "categorical_features": categorical_features,
    "task": "classification"
}

joblib.dump(
    artifact,
    "/content/ml_streamlit_project/model.joblib"
)
```

El artefacto contiene tanto el modelo como información necesaria para construir la interfaz.

# 9. Ruta B - Construcción no supervisada completa

Para clustering recomendamos trabajar inicialmente con variables numéricas interpretables. Si existen categóricas, pueden incorporarse mediante codificación, pero la interpretación de distancias requiere mayor cuidado.

## Paso 9.1 - Definir variables

```
numeric_features = [
    "edad",
    "ingreso",
    "frecuencia_compra",
    "ticket_promedio"
]

features = numeric_features
X = df[features].copy()
```

## Paso 9.2 - Preprocesamiento + K-Means

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("cluster", KMeans(
        n_clusters=3,
        random_state=42,
        n_init="auto"
    ))
])

labels = model.fit_predict(X)
```

## Paso 9.3 - Evaluar estructura

```
from sklearn.metrics import silhouette_score

X_processed = model[:-1].transform(X)

score = silhouette_score(X_processed, labels)
print("Silhouette:", score)
```

No existe una métrica única que «demuestre» que los clusters son correctos. La pareja debe combinar evidencia cuantitativa con interpretabilidad.

# 10. Ruta B - Interpretar los clusters

Un cluster llamado 0, 1 o 2 no tiene significado por sí mismo. Debemos perfilarlo.

```
df_clustered = df.copy()
df_clustered["cluster"] = labels

profiles = (
    df_clustered
    .groupby("cluster")[numeric_features]
    .mean()
    .round(2)
)

display(profiles)
```

A partir de esta tabla, la pareja puede crear nombres como «alta frecuencia / alto ticket» o «bajo ingreso / baja frecuencia». Estos nombres deben surgir de los datos, no inventarse antes del análisis.

## Guardar perfiles y modelo

```
cluster_descriptions = {
    0: "Describa aquí el perfil del cluster 0",
    1: "Describa aquí el perfil del cluster 1",
    2: "Describa aquí el perfil del cluster 2",
}

artifact = {
    "model": model,
    "features": features,
    "numeric_features": numeric_features,
    "task": "clustering",
    "cluster_descriptions": cluster_descriptions
}

joblib.dump(
    artifact,
    "/content/ml_streamlit_project/model.joblib"
)
```

# 11. Convertir el entrenamiento en un archivo train.py

Hasta ahora podemos haber experimentado en celdas. El siguiente paso es transformar el procedimiento final en un script reproducible.

### Estructura sugerida de train.py

```
%%writefile /content/ml_streamlit_project/train.py
from pathlib import Path
import joblib
import pandas as pd

DATA_PATH = Path("data/datos.csv")
MODEL_PATH = Path("model.joblib")

def load_data() -> pd.DataFrame:
    return pd.read_csv(DATA_PATH)

def train_model(df: pd.DataFrame):
    # 1. Definir variables
    # 2. Construir preprocesamiento
    # 3. Entrenar
    # 4. Evaluar
    # 5. Construir artifact
    return artifact

def main() -> None:
    df = load_data()
    artifact = train_model(df)
    joblib.dump(artifact, MODEL_PATH)
    print(f"Modelo guardado en {MODEL_PATH}")

if __name__ == "__main__":
    main()
```

Cada pareja debe reemplazar el bloque interno por su solución supervisada o no supervisada. El objetivo es que el modelo pueda reconstruirse ejecutando python train.py.

# 12. Preparar el dataset dentro del proyecto

Si el dataset puede incluirse en el repositorio, cree una carpeta data/ y copie allí el archivo. Para datasets privados o grandes, el README debe explicar cómo obtenerlos sin versionar información sensible.

```
from pathlib import Path
import shutil

data_dir = Path("/content/ml_streamlit_project/data")
data_dir.mkdir(exist_ok=True)

# Ejemplo: si el archivo cargado se llama datos.csv
shutil.copy("datos.csv", data_dir / "datos.csv")
```

No suba información personal, confidencial o restringida a un repositorio público.

# 13. Crear app.py - interfaz adaptable al artefacto

```
%%writefile /content/ml_streamlit_project/app.py
import joblib
import pandas as pd
import streamlit as st

artifact = joblib.load("model.joblib")

model = artifact["model"]
features = artifact["features"]
numeric_features = artifact.get("numeric_features", [])
categorical_features = artifact.get("categorical_features", [])
task = artifact["task"]

st.set_page_config(
    page_title="ML App",
    page_icon="□"
)

st.title("Aplicación de Machine Learning")
st.write("Ingrese las características del nuevo caso.")
```

```
values = {}

for feature in numeric_features:
    values[feature] = st.number_input(
        feature,
        value=0.0
    )

for feature in categorical_features:
    values[feature] = st.text_input(feature)

if st.button("Ejecutar modelo"):
    X_new = pd.DataFrame(
        [values],
        columns=features
    )

    result = model.predict(X_new)[0]

    if task == "classification":
        st.success(f"Clase predicha: {result}")

        if hasattr(model, "predict_proba"):
            probabilities = model.predict_proba(X_new)[0]
            st.write("Probabilidades:", probabilities)

    elif task == "clustering":
        descriptions = artifact[
            "cluster_descriptions"
        ]
        st.success(f"Cluster asignado: {result}")
        st.info(descriptions.get(
            int(result),
            "Sin descripción"
        ))
```

Esta versión permite conservar una arquitectura común para las dos rutas. Cada pareja puede mejorar la interfaz usando selectbox, sliders, validaciones y textos explicativos.

# 14. Mejorar las entradas categóricas

Para una aplicación más robusta, en lugar de escribir libremente una categoría conviene guardar sus valores permitidos durante entrenamiento y usar un selectbox.

```
category_options = {
    col: sorted(
        df[col].dropna().astype(str).unique().tolist()
    )
    for col in categorical_features
}

artifact["category_options"] = category_options
```

Y en Streamlit:

```
category_options = artifact.get(
    "category_options",
    {}
)

for feature in categorical_features:
    values[feature] = st.selectbox(
        feature,
        category_options[feature]
    )
```

Esto reduce errores y hace que la app sea consistente con los valores observados durante entrenamiento.

# 15. Probar Streamlit en Google Colab

Instalar Streamlit ya se realizó al comienzo. Ahora levantamos la aplicación en segundo plano.

```
!streamlit run   /content/ml_streamlit_project/app.py   --server.port 8501   --server.address 0.0.0.0   > /content/
streamlit.log 2>&1 &
```

Comprobar que inició:

```
!cat /content/streamlit.log
```

Para visualizarla desde el navegador se puede usar un túnel temporal. Una opción sencilla para el laboratorio es localtunnel.

```
!npm install -g localtunnel
```

```
!npx localtunnel --port 8501
```

El comando mostrará una URL temporal. Esta URL sirve para probar la app durante el desarrollo; no es el despliegue final.

Si la red institucional bloquea el túnel, la práctica puede continuar: revisen que el log indique que Streamlit inició correctamente y continúen con GCP.

# 16. Pruebas mínimas antes de Docker

| | |
|---|---|
| 1 | Ejecute nuevamente train.py y compruebe que model.joblib se genera. |
| 2 | Abra Streamlit y pruebe al menos tres casos diferentes. |
| 3 | Compruebe que las columnas de entrada coinciden con las usadas durante entrenamiento. |
| 4 | Pruebe valores extremos o categorías válidas distintas. |
| 5 | Reinicie Streamlit y verifique que la aplicación carga model.joblib sin volver a entrenar. |
| 6 | Documente una entrada de ejemplo y su salida esperada en README.md. |

# 17. Archivos para Docker

## requirements.txt

```
%%writefile /content/ml_streamlit_project/requirements.txt
streamlit
pandas
numpy
scikit-learn
joblib
openpyxl
```

## .dockerignore

```
%%writefile /content/ml_streamlit_project/.dockerignore
.git
.venv
__pycache__
*.pyc
.ipynb_checkpoints
.env
*.log
```

## Dockerfile

```
%%writefile /content/ml_streamlit_project/Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8501
```

```
CMD ["streamlit", "run", "app.py",
     "--server.address=0.0.0.0",
     "--server.port=8501"]
```

El Dockerfile se crea en Colab como un archivo de texto, pero no necesitamos ejecutar Docker dentro de Colab. La imagen se construirá en la VM Linux.

# 18. ¿Qué significa cada instrucción Docker?

| Instrucción | Significado |
|---|---|
| FROM python:3.12-slim | Define una imagen base pequeña que ya contiene Python. |
| WORKDIR /app | Define el directorio de trabajo dentro de la imagen. |
| COPY requirements.txt . | Copia primero las dependencias para aprovechar el cache de capas. |
| RUN pip install ... | Instala las librerías dentro de la imagen. |
| COPY . . | Copia el proyecto completo. |
| EXPOSE 8501 | Documenta el puerto utilizado por Streamlit. |
| CMD ... | Define el proceso que se ejecuta al iniciar el contenedor. |

# 19. Estructura final del proyecto

```
ml_streamlit_project/
|
|-- app.py
|-- train.py
|-- model.joblib
|-- requirements.txt
|-- Dockerfile
|-- .dockerignore
|-- README.md
|
`-- data/
    `-- datos.csv
```

Si el dataset no puede publicarse, omita el archivo y explique en README cómo debe suministrarse.

# 20. README mínimo obligatorio

- Título y objetivo del proyecto.
- Descripción del dataset y unidad de observación.
- Ruta elegida: supervisada o no supervisada.
- Variables utilizadas.
- Preparación de datos.
- Modelo y justificación.
- Métrica(s) o criterio de evaluación.
- Interpretación de la salida.
- Cómo ejecutar train.py.
- Cómo ejecutar Streamlit.
- Cómo construir la imagen Docker.
- Cómo ejecutar el contenedor.
- Ejemplo de entrada y salida.
- Autores de la pareja.

# 21. Guardar el proyecto en GitHub

La opción más sencilla en un laboratorio restringido es crear el repositorio desde la web de GitHub y subir los archivos mediante el navegador.

Antes de subir, puede comprimir el proyecto desde Colab:

```
!cd /content && zip -r ml_streamlit_project.zip ml_streamlit_project
```

También puede descargarlo:

```
from google.colab import files
files.download("/content/ml_streamlit_project.zip")
```

Después, cree el repositorio y suba los archivos descomprimidos. No publique credenciales, tokens, .env o datos sensibles.

# 22. Crear la máquina virtual en Google Cloud

| | |
|---|---|
| 1 | Entrar a Google Cloud Console desde el navegador. |
| 2 | Crear o seleccionar el proyecto destinado al laboratorio. |
| 3 | Abrir Compute Engine. |
| 4 | Habilitar Compute Engine API si la consola lo solicita. |
| 5 | Crear una instancia de VM. |
| 6 | Seleccionar Linux; Ubuntu 24.04 LTS es apropiado para la práctica. |

| | |
|---|---|
| 7 | Elegir un tamaño pequeño suficiente para una aplicación educativa y revisar el costo mostrado. |
| 8 | Confirmar que la instancia tendrá una dirección IP externa. |
| 9 | Crear la VM. |
| 10 | Usar el botón SSH de la consola para abrir una terminal directamente en el navegador. |

La VM puede generar costos mientras exista o esté ejecutándose. Al terminar el laboratorio, detenga o elimine los recursos que ya no necesite.

# 23. Instalar Docker y Git en la VM

```
sudo apt update
sudo apt install -y docker.io git

sudo systemctl enable --now docker

docker --version
git --version
```

Opcionalmente, para ejecutar Docker sin sudo:

```
sudo usermod -aG docker $USER
```

Después cierre la sesión SSH y vuelva a entrar. Si no desea hacer esto, puede anteponer sudo a los comandos Docker.

# 24. Clonar el proyecto en la VM

```
git clone <URL_DEL_REPOSITORIO>
cd <NOMBRE_DEL_REPOSITORIO>

ls -la
```

Compruebe que aparecen Dockerfile, app.py, model.joblib y requirements.txt.

Si el repositorio es privado, utilice un método de autenticación permitido por la Universidad y nunca escriba un token dentro del Dockerfile o del código.

# 25. Construir la imagen Docker

```
docker build -t ml-streamlit-app .
```

El punto final significa «usar el directorio actual como contexto de construcción». Docker leerá el Dockerfile, descargará la imagen base, instalará dependencias y copiará el proyecto.

Verificar la imagen:

```
docker images
```

Conceptualmente:

```
Dockerfile  --docker build-->  Imagen

Imagen      --docker run---->  Contenedor
```

Una imagen no es todavía la aplicación ejecutándose. El contenedor sí es un proceso en ejecución basado en esa imagen.

# 26. Ejecutar el contenedor

Primera prueba interactiva:

```
docker run --rm -p 8501:8501 ml-streamlit-app
```

Si no hay errores, deténgalo con Ctrl+C y ejecútelo en segundo plano:

```
docker run -d   --name ml-streamlit   --restart unless-stopped   -p 8501:8501   ml-streamlit-app
```

Verificar:

```
docker ps
docker logs ml-streamlit
```

-p 8501:8501 conecta el puerto 8501 de la VM con el puerto 8501 del contenedor. Streamlit debe escuchar en 0.0.0.0 para aceptar conexiones externas.

# 27. Abrir el puerto 8501 en Google Cloud

El contenedor puede estar funcionando y aun así no ser accesible desde Internet. GCP controla el tráfico mediante reglas de firewall.

Una estrategia didáctica es asignar a la VM una etiqueta de red, por ejemplo streamlit-server, y crear una regla para TCP 8501.

```
gcloud compute firewall-rules create allow-streamlit-8501   --allow=tcp:8501   --direction=INGRESS   --target-tags=
streamlit-server
```

La etiqueta streamlit-server debe estar aplicada a la VM. La regla puede crearse también desde la interfaz gráfica de VPC/Firewall.

Para un laboratorio temporal puede permitirse acceso público. En un despliegue real deben considerarse HTTPS, autenticación, restricción de orígenes y una arquitectura frontal adecuada.

# 28. Abrir la aplicación

Copie la IP externa de la VM y abra:

```
http://IP_EXTERNA_DE_LA_VM:8501
```

Si aparece la interfaz de Streamlit y puede generar una predicción/asignación, el flujo completo está funcionando.

# 29. Diagnóstico paso a paso

| Situación | Comando / revisión |
|---|---|
| train.py falla | Revise ruta del dataset, nombres de columnas, tipos y dependencias. |
| model.joblib no existe | Confirme que joblib.dump se ejecutó y revise la ruta de salida. |
| Streamlit falla en Colab | Revise !cat /content/streamlit.log. |
| docker build falla | Lea el error completo; revise requirements.txt, Dockerfile y archivos copiados. |
| Contenedor se detiene | docker ps -a y docker logs ml-streamlit. |
| Puerto no responde | Confirme -p 8501:8501 y --server.address=0.0.0.0. |
| Funciona en VM pero no fuera | Revise firewall, etiqueta de red e IP externa. |
| Cambios no aparecen | git pull -> detener/eliminar contenedor -> docker build -> docker run. |

# 30. Cómo actualizar una aplicación desplegada

Si la pareja cambia app.py o el modelo y actualiza GitHub, en la VM puede ejecutar:

```
git pull

docker stop ml-streamlit
docker rm ml-streamlit

docker build -t ml-streamlit-app .

docker run -d   --name ml-streamlit   --restart unless-stopped   -p 8501:8501   ml-streamlit-app
```

Esto muestra una idea importante: el contenedor no «se actualiza solo». Se construye una nueva imagen y se crea un nuevo contenedor.

# 31. Prueba cruzada entre parejas

Una pareja diferente debe reproducir el proyecto siguiendo únicamente el README.

| | |
|---|---|
| 1 | Recibir únicamente la URL del repositorio. |
| 2 | Clonar el repositorio. |
| 3 | Leer README sin instrucciones verbales adicionales. |
| 4 | Construir la imagen. |
| 5 | Ejecutar el contenedor. |
| 6 | Probar al menos un caso. |
| 7 | Registrar cualquier instrucción faltante o error de reproducibilidad. |

La prueba cruzada convierte la reproducibilidad en un criterio observable, no en una afirmación.

# 32. Entregables

- Repositorio GitHub.
- Dataset o instrucciones claras para obtenerlo.
- train.py reproducible.
- model.joblib.
- app.py funcional.
- requirements.txt.
- Dockerfile y .dockerignore.
- README completo.
- Aplicación desplegada en la VM durante la demostración.
- Breve evidencia de la prueba cruzada con otra pareja.

- Interpretación de resultados: desempeño predictivo en supervisado o perfiles de clusters en no supervisado.

## 33. Rúbrica sugerida

| Criterio | Peso | Qué se espera |
|---|---|---|
| Problema y datos | 10% | Objetivo claro, dataset pertinente y variables justificadas. |
| Modelamiento | 20% | Pipeline reproducible, evaluación correcta e interpretación. |
| Streamlit | 15% | Entradas coherentes, salida interpretable y app funcional. |
| Docker | 25% | Dockerfile correcto, imagen reproducible y contenedor funcional. |
| GCP | 15% | Despliegue funcional y acceso durante la demostración. |
| README y prueba cruzada | 15% | Otra pareja puede reproducir el proyecto sin ayuda adicional. |

## 34. Checklist final de la pareja

- Entendemos qué representa una fila del dataset.
- Definimos claramente si la ruta es supervisada o no supervisada.
- Seleccionamos variables justificadas.
- El preprocesamiento está incluido en el pipeline.
- train.py puede ejecutarse de principio a fin.
- model.joblib se genera correctamente.
- app.py no vuelve a entrenar el modelo.
- Probamos Streamlit en Colab.
- Creamos requirements.txt, Dockerfile y .dockerignore.
- El repositorio contiene todos los archivos necesarios.
- La VM está creada y podemos acceder por SSH.
- Docker está instalado.
- docker build termina correctamente.
- docker run deja el contenedor activo.
- docker logs no muestra errores críticos.
- La regla de firewall está configurada.
- La aplicación abre usando la IP externa.
- Otra pareja pudo reproducir el proyecto.
- Sabemos detener/eliminar la VM al finalizar para controlar costos.

## 35. Preguntas para la discusión final

- ¿Qué diferencia existe entre el notebook de experimentación y train.py?
- ¿Por qué conviene guardar preprocesamiento y modelo dentro del mismo Pipeline?
- ¿Qué ocurre si Streamlit transforma los datos de una forma distinta al entrenamiento?
- ¿Qué diferencia existe entre model.joblib y la imagen Docker?
- ¿Qué contiene realmente una imagen Docker?
- ¿Qué diferencia existe entre una imagen y un contenedor?
- ¿Por qué escribimos el Dockerfile en Colab pero lo construimos en la VM?
- ¿Por qué necesitamos mapear puertos?
- ¿Por qué existe además una regla de firewall en GCP?
- ¿Qué parte del flujo cambia entre clasificación y clustering?
- ¿Qué parte del flujo sería prácticamente idéntica si el modelo fuera una red neuronal?
- ¿Qué mejoraríamos para convertir este laboratorio en un despliegue productivo?

## 36. Extensiones opcionales

- Guardar la imagen en un container registry.
- Usar GitHub Actions para construir automáticamente la imagen.
- Configurar Nginx como reverse proxy.
- Agregar HTTPS.
- Agregar autenticación.
- Separar frontend y API.
- Crear una API con FastAPI y usar Streamlit únicamente como cliente.
- Comparar la VM con Cloud Run.
- Registrar versiones de modelos y métricas.
- Agregar tests antes de construir la imagen.

Mensaje central del reto: un modelo útil no termina en model.fit(). Debemos ser capaces de reconstruirlo, servirlo, empaquetarlo, ejecutarlo en otro entorno y explicar cómo otra persona puede reproducir todo el proceso.

Nota técnica: el túnel de Colab se utiliza únicamente como mecanismo temporal de prueba. Docker se ejecuta en la VM, donde los estudiantes pueden observar explícitamente el sistema Linux, la construcción de la imagen, el contenedor, los puertos y los logs.